Exact Bidirectional Steganographic Transport with Language and Pixel Autoregressive Models

Keywords: Generative Steganography, Autoregressive Models, Exact Recovery, Steganalysis, GPU Inference.

Abstract: Generative steganography must survive the serialization of a model's decisions into a delivered file. We develop a shared authenticated packet protocol for hiding canonical grayscale images in generated text and literal UTF-8 messages in generated PNGs, with recovery by an independent receiver using only the saved artifact and shared configuration, context and key. A controlled comparison of fixed-rank, entropy-gated and arithmetic coding uses a language model and a pixel autoregressive model under the same packet and modality-specific carrier limits. Across 120 held-out transmissions, fixed and gated coding each recover 20/20 payloads in both directions. Arithmetic coding recovers 20/20 PNG payloads but 0/20 text payloads because the generated trajectories provide insufficient information before the token cap. Known-model detection exposes substantial distributional differences. Rescoring the same PNGs with one frozen alternate conditioning row reduces fixed-rank mean-surprisal AUC from 0.8850 to 0.6975, without establishing secrecy. Finally, CUDA graph replay reduces matched development encode-plus-decode latency by 5.13 times while preserving checked probabilities, carrier bytes and recovery. These results connect exact artifact transport, finite capacity, observer knowledge and practical execution cost.

1 INTRODUCTION

Autoregressive models carry information through choices made during generation. A language model can select a token according to a hidden symbol, and a pixel model can do the same with a channel value. The receiver must recover the intended bytes from the object actually delivered. A text file does not preserve token identifiers, and a PNG contains pixels rather than the sender's latent state or probability trace.[1]

Calgacus proposes cross-domain rank transfer, RankCloak develops bounded byte coding and entropy gating, and Pixel-Stega carries bits through autoregressive images (Norelli and Bronstein, 2026; Mantzaris, 2026b; Zhang et al., 2021). We connect these foundations through a common authenticated protocol for bidirectional recovery from saved artifacts. This makes coding, serialization and finite capacity jointly testable across text and pixel models.

Our framework transports canonical images or literal messages in the same fixed-size encrypted packet within each payload group. Text generation enforces complete-prefix tokenization consistency. Image generation uses the discrete conditional probabilities of observable RGB values. Fresh receivers reconstruct model state from saved files, and a separate evaluator checks source-byte equality. Under this contract, we compare fixed-rank, gated and arithmetic coding, measure detection with a declared observer, and test one conditioning mismatch by rescoring the same PNGs. Exact probability-stream and output comparisons then establish a GPU execution improvement.

The prospective study comprises 20 held-out payload groups per direction and three methods, giving 120 transmissions. Fixed and gated coding recover every payload. Arithmetic A1 recovers all PNG payloads but accumulates insufficient information for any complete text packet under the 2,048-token cap. A matched development benchmark reduces image encode-plus-decode latency from 177.82 to 34.64 seconds while preserving the tested outputs. Together, the shared protocol and controlled comparisons explain how artifact reconstruction, distributional change and execution cost determine practical generative transport.

2 RELATED WORK

2.1 Distributional and Rank-Based Coding

Neural linguistic steganography uses an autoregressive distribution as an information-bearing alphabet. Neural Linguistic Steganography (Ziegler et al., 2019) encodes a message through arithmetic decoding and recovers it by replaying arithmetic encod-

[1] Drafting used Codex (OpenAI, 2025).

ing. That work connects information efficiency to the model's conditional distributions. Our comparator follows that approach with an explicitly identified finite-message adaptation.[2]

The coupling formulation (Schroeder de Witt et al., 2023) characterizes perfectly secure procedures and evaluates minimum entropy coupling on language, audio and Image Transformer channels. More recent rotation-based range coding (Yan and Murawaki, 2026) offers distribution-preservation guarantees and high entropy utilization. These approaches pursue distribution preservation, beyond packet recovery. They were not experimentally matched here, so their reported guarantees are distinct from our empirical comparisons.

Calgacus (Norelli and Bronstein, 2026) instead transfers each payload token's context-dependent rank to a cover context. It discusses separate models, different vocabulary sizes and other discrete domains. RankCloak (Mantzaris, 2026b) adapts rank transport to serialized cryptographic artifacts and implements bounded byte coding and entropy gating. Its repository manuscript reports 6,480 recoveries with preserved token sequences, but 88/144 in a rendered-text subset. These reported results motivate the artifact endpoint rather than form a matched baseline. LlmStenoExplore (Mantzaris, 2026a) reproduces Calgacus and investigates finite-key collisions and perturbation sensitivity. We adapt RankCloak's rank, replay and numerical practices to saved-artifact recovery, with the packet key separate from model conditioning.

2.2 Pixels and Tokenization

Pixel-Stega (Zhang et al., 2021) is the closest image precedent. It obtains autoregressive pixel probabilities from PixelCNN++, applies finite-precision arithmetic stegosampling, and describes transmission through a lossless public channel. Its color-image formulation identifies the red channel as the embedded variable. We extend the evaluated contract to successive observable R, G and B values with conditional mixture posteriors. Three coders share one authenticated packet contract across text and PNG carriers, with a shared-row canvas for PNG. Pixel-Stega's published capacity and detector results use different allocations and are not matched to our net authenticated rates.

Published stepwise verification (Yan and Murawaki, 2025) directly addresses detokenization and retokenization inconsistency through stepwise candidate verification. We adapt this existing idea to a pinned byte-exact tokenizer and a top-256 candidate pool. A paired development comparison isolates replacing singleton eligibility with complete-prefix consistency while holding the other rules fixed.

2.3 Detectability and Execution

GLTR (Gehrmann et al., 2019) exposes model-based probability and rank statistics to support generated-text analysis. Our observer instead distinguishes steganographic carriers from ordinary samples of the same eligible distribution using two frozen whole-carrier scores. The context experiment changes the observer's row while keeping the exact image model known.

CUDA graphs are established execution infrastructure (PyTorch Contributors, 2024). We measure their benefit for repeated PixelCNN++ replay and verify exact coder-interface and saved-file equivalence. A faster but numerically changed distribution could reorder ranks and break recovery.

3 METHOD

3.1 A Shared Artifact Contract

A public profile specifies the model, observable representation, numerical rules, coder and budget. The sender seals canonical source bytes once and encodes the packet into carrier y. The receiver obtains only saved y, the profile, shared context and encryption key. Source bytes/digests, sender identifiers, ranks, packets and latent codes are excluded. Figure 1 shows both directions using the first manifest-ordered fixed-rank example in each direction. Panel A carries canonical image pixels through saved text. Panel B carries literal text through a saved PNG.[3]

Images are canonical 16×16 grayscale arrays of unsigned eight-bit pixels, 256 bytes in row-major order. Exact recovery restores this preprocessed array, not the larger original image or its container. Text is 32–128 literal UTF-8 bytes without normalization, added newlines or replacement decoding.

The plaintext consists of an eight-byte header followed by a 256-byte payload slot. In order, the unsigned header fields are version (one byte, value 1), kind (one byte, 1 for text or 2 for grayscale), length (two bytes), width (two bytes) and height (two bytes). Multibyte fields are big-endian. Text dimensions are zero, and image dimensions are 16 by 16. The source occupies the beginning of the slot, followed by zero

[2] Drafting used Codex (OpenAI, 2025).

[3] Drafting used Codex (OpenAI, 2025).

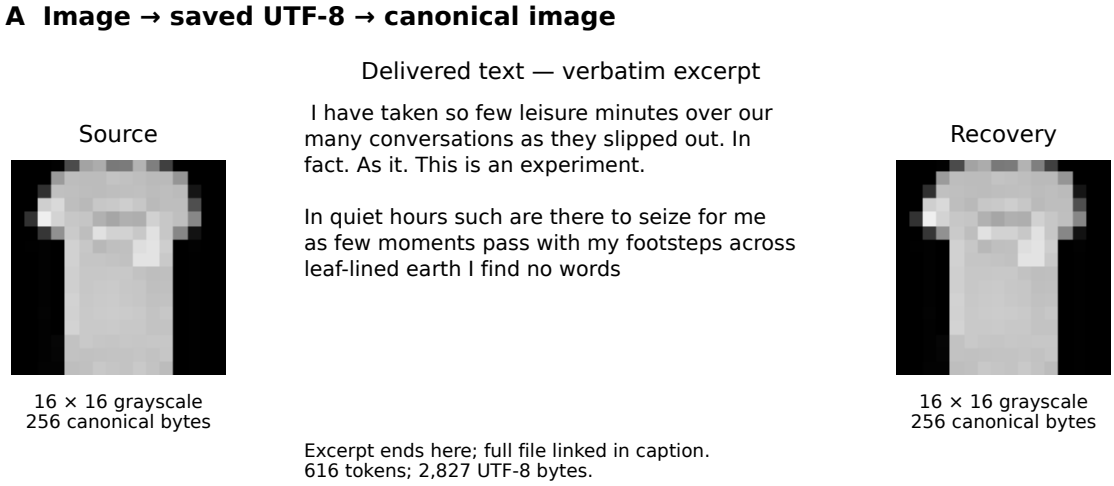

Figure 1: Actual bidirectional transport, with examples selected by manifest order rather than appearance. A shows a canonical 16×16 grayscale image, a verbatim excerpt of its saved 616-token UTF-8 carrier and recovery of all 256 pixel bytes, not an arbitrary original image file. The excerpt is incomplete. The figure's full-file note refers to locally retained material, not an available review link. B shows the complete 63-byte source text, its delivered 32×31 RGB PNG and exact text-byte recovery. Real pixels use nearest-neighbor enlargement with preserved aspect ratios. Each fresh receiver uses only its carrier, model/profile, shared context and key. The text direction uses the fixed prompt. For PNG, the withheld 96-byte conditioning row is prepended internally to form a 32×32 canvas. Source equality is checked separately after decoding.

bytes. There is no MIME string or payload-specific sidecar.

AES-256-GCM (Dworkin, 2007) authenticates and encrypts the entire header and slot using a fresh random key and a unique random 12-byte nonce. The associated data are the exact ASCII bytes `ImageCalgacus/packet/v1`, with no newline. The carried packet is

$$\begin{aligned} P &= \text{nonce} \,\|\, \text{ciphertext} \,\|\, \text{tag}, \\ |P| &= 12 + 264 + 16 = 292 \text{ bytes}. \end{aligned} \tag{1}$$

The known 2,336-bit target avoids circular framing around an encrypted length. After authentication the parser validates type, length, dimensions, UTF-8 and all padding bytes, returning no plaintext on failure. The evaluator checks equality separately. Authenticated encryption protects content under its usual assumptions, not the channel's existence.

One immutable packet is shared across the paired methods in a payload/context group. Reusing that ciphertext is not a new encryption under a reused nonce. Keys and nonces come from operating-system randomness, independently of sampling seeds. Private packet bindings support continuation without resealing an existing group or supplying packet state to its receivers.

3.2 Observable Conditional Distributions

At position t, the backend supplies eligible symbols E_t, normalized probabilities $q_t(v)$ and a deterministic rank order. Conditioning includes the shared context and all preceding observed symbols, including skips and completion. Text ranks use descending logits, with ascending token identifier breaking exact ties. Image ranks use descending log probability, then ascending channel value. Probability bookkeeping uses float64, stable normalization and ascending-identifier summation. NaN, positive infinity, empty support and invalid normalization are explicit failures. Numerical zero mass is excluded and the remaining distribution renormalized, without an added probability floor. Ordinary sampling is temperature-one inverse-CDF sampling in identifier order, with no nucleus truncation.

For text, the model is Llama 3 8B Instruct (Meta, 2024), not an image model. Let T tokenize bytes without BOS insertion or special-token interpretation, and let D return exact bytes without cleanup. Given carrier prefix s, each candidate v must satisfy

$$T\big(D(s \,\|\, [v])\big) = s \,\|\, [v]. \tag{2}$$

The pool contains the top 256 model tokens *before* filtering. Special or control tokens, empty renderings and invalid UTF-8 are excluded. Equation 2 is checked on the complete carrier prefix, not a prompt/carrier concatenation. The model sees one BOS followed by separately tokenized prompt bytes and carrier identifiers, without a chat template. The receiver tokenizes the delivered file separately, checks exact byte round-trip equality, and replays the same prefix tests. Payload positions, gate skips, arithmetic steps and the 32-token ordinary tail all obey this rule.

For PNG carriers, PixelCNN++ (Salimans et al., 2017) models a native 32×32 RGB canvas. A shared 1×32 RGB row occupies the first row. Only the remaining 31 rows are delivered, giving 992 pixels or 2,976 channel values. Each channel value in $\{0, \ldots, 255\}$ is observable. Generation and replay use raster order, with R then G then B within each pixel. The receiver reads an eight-bit RGB PNG without palette conversion, alpha compositing or color transformations and prepends the shared row internally.

The network predicts ten mixture weights, channel means, log scales and RGB dependency coefficients per pixel, not 256 independent categorical logits. Let $f_{k,C}$ denote a discretized logistic component mass for channel C. For normalized intensity $z(v) = 2v/255 - 1$, component probabilities are logistic CDF differences over half-bin width $1/255$, with infinite tails at 0 and 255. Log scales are lower bounded by -7 and dependency coefficients use tanh. We evaluate the discrete masses in float64 rather than using the upstream continuous sampler or its small-density fallback.

The channels require posterior updates, not independent mixture marginals. For mixture weights w_k and observed red value r,

$$q_R(r) = \sum_k w_k f_{k,R}(r), \tag{3}$$

$$w_k^R = \frac{w_k f_{k,R}(r)}{q_R(r)}, \tag{4}$$

$$q_G(g \mid r) = \sum_k w_k^R f_{k,G}(g \mid r). \tag{5}$$

The green mean is shifted by $a_{RG} z(r)$. After green, $w_k^{RG} \propto w_k^R f_{k,G}(g \mid r)$, and blue uses these normalized weights with mean shift $a_{RB} z(r) + a_{GB} z(g)$. This yields the joint RGB mixture through the product of the three conditionals. One network forward per visible pixel supplies parameters reused within that pixel. Causal checks establish that unobserved pixels do not alter these parameters.

3.3 Three Packet Coders

Fixed rank. Each packet byte yields its high nibble and then low nibble. A nibble $d \in \{0, \ldots, 15\}$ selects eligible rank $d + 1$. Exactly $292 \times 2 = 584$ positions carry the packet. The receiver inverts observed ranks into bytes and stops by this fixed count. Fewer than 16 eligible symbols while the packet is pending causes a support failure, not a smaller alphabet.

Entropy-gated rank. The same four-bit operation is applied only if

$$H(q_t) = -\sum_{v \in E_t} q_t(v) \log_2 q_t(v) > \tau. \tag{6}$$

Otherwise the sender samples ordinarily and consumes no packet bits. The receiver recomputes the strict comparison from the observed prefix, so no gate trace is transmitted. Separate thresholds were calibrated from ordinary development traces and frozen at 0.5983972524635524 bits for text and 3.2159513527579326 bits for channels. They were not adjusted after failures. Gating is inherited from RankCloak, with this protocol's strict comparison and common eligible distribution.

Arithmetic A1. We independently implement a finite-precision arithmetic comparator following Neural Linguistic Steganography (Ziegler et al., 2019).

A1 identifies our framing adaptation, not a new security construction. It starts with the half-open interval $[0, 2^{32})$. At width R, retain symbols with $q_t(v) \geq 1/R$ and at least the top $\min(2, |E_t|, R)$ symbols. Renormalize retained probabilities, round bin widths to nearest integer with ties to even, truncate before the first cumulative overflow, and assign residual width to the first symbol. Remove zero-width bins. Positive widths must sum to R.

The next 32 source bits, zero-extended only beyond the packet, select the interval bin. Sender and receiver emit the common leading bits of its lower bound and upper bound minus one, then shift the interval accordingly. A zero-bit step updates the interval and consumes carrier capacity. Packet completion occurs at the first common-prefix emission reaching 2,336 bits. At most 31 emitted suffix bits are checked to be zero and discarded. There is no end token, final-artifact endpoint dump or implicit flush during ordinary completion. The receiver needs no sender offset to determine progress.

This framing differs from the inspected reference's final-token endpoint handling. A1 has a separately demonstrated finite-precision stagnation case and no universal termination guarantee. We keep its precision, partitioning and stopping rules fixed throughout the study. Capacity exhaustion is a legitimate result, whereas sender/receiver desynchronization is an implementation defect. An arithmetic packet prefix, even when verified against private sender diagnostics, is not authenticated payload delivery.

3.4 Stopping, Completion and Conditional Inversion

All text carriers have a 2,048-token cap. After packet completion the sender must add exactly 32 ordinary tokens within that cap. A packet completing too late for the tail is packet-complete but carrier-incomplete. An unfinished packet is not truncated early to reserve a tail. PNG generation always completes all 2,976 channels after the packet stop, including any remainder of a partially used RGB pixel. No packet is stored in PNG metadata or appended bytes.

Under identical model state, numerical ordering and reconstructible observations, inversion follows by induction. At each position the receiver computes the same eligible distribution and coder transition as the sender. It therefore recovers the same nibble, skip or arithmetic common prefix, and observes the same symbol before the next transition. Equation 2 supplies the text reconstruction invariant. Lossless pixel equality supplies the PNG invariant. The known packet size supplies framing. Sufficient support and successful finite termination remain necessary conditions, rather than consequences of this argument.

Senders exit before fresh receivers start. Their input directories contain only carrier, profile, context and key, with model assets separately available. Evaluation follows receiver exit. Reports distinguish packet and carrier completion, receiver recovery, authentication, carrier conformance and source equality. This data-flow separation is not an adversarial operating-system sandbox.

4 EXPERIMENTAL DESIGN

4.1 Sources, Contexts and Pairing

Development used 20 images and 20 texts with two contexts per direction. Five synthetic image fixtures were supplemented by 15 Fashion-MNIST training images. Five literal text fixtures, including non-ASCII UTF-8, were supplemented by 15 narrative spans from Gutenberg ebooks 11 and 84. Sources, canonicalization, contexts, profiles and thresholds were frozen before prospective generation.[4]

The held-out image set contains 20 Fashion-MNIST test images, two per class (Xiao et al., 2017), BOX-resized from 28×28 grayscale to 16×16. Twenty nonoverlapping text spans come from Austen's *Pride and Prejudice* and Doyle's *The Adventures of Sherlock Holmes*, Gutenberg ebooks 1342 and 1661. Ten messages occupy each 32–64-byte and 65–128-byte band. Observed lengths are 58–128 bytes, with mean 93.2. Snapshot hashes, source indices/byte offsets and exclusions are recorded. Boilerplate, duplicate original/canonical bytes and development/held-out overlap are excluded without using generated outcomes. Fashion-MNIST's MIT notice and Gutenberg attribution are retained. The texts are unrestricted under US copyright, not necessarily every jurisdiction.

The prospective prompt is "Write a calm field-journal entry about a walk through a temperate forest. Use continuous prose." Row1 is the first row of an independently generated development image with context seed 2001. Neither context is included in the delivered carrier. Table 1 summarizes the frozen allocation.

The allocation is $20 \times 3 \times 2 = 120$ stego units. Each payload/context group shares one immutable packet, source and model across three methods. Sampling seeds are separate from encryption randomness.

[4]Drafting used Codex (OpenAI, 2025).

Table 1: Frozen prospective specification. Dimensions are width by height. Both directions share a 292-byte packet, but carrier symbols and useful-rate units differ. One sealed packet is shared across methods within each payload/context group.

Property	Image payload in text	Text payload in PNG
Canonical source	16×16 grayscale, 256 raw bytes	Literal UTF-8, 32–128 bytes
Delivered artifact	Strict UTF-8, at most 2,048 tokens	32×31 RGB8 PNG, 2,976 channels
Neural model	Llama 3 8B Instruct, Q4_K_M GGUF	CIFAR-10 pretrained PixelCNN++
Runtime	llama-cpp-python 0.3.23, 33/33 CUDA layers	PyTorch 2.5.1+cu124, float32 CUDA
Shared context	Forest prompt, separate tokenization	Private 1×32 RGB row, 96 bytes
Eligible symbols	Top 256, then complete-prefix filtering	Discrete R/G/B values 0–255
Coders	Fixed radix 16, gated radix 16, arithmetic A1	Same coders and 2,336-bit target
Completion	32 ordinary tokens after the packet	All channels through the last visible pixel
Payload groups	20 held-out images, one context	20 held-out texts, one context
Controls	20 traces, matched prefixes	20 PNGs shared across methods

Terminal failures remain observations, and continuation neither duplicates completed carriers nor replaces difficult sources.

One independent ordinary control is generated per group. Text controls extend to the group's longest realized carrier, 2,048 tokens in every prospective group, with length-matched prefixes for fixed and gated methods. PNG methods share one full image. All controls use the same eligibility and ordinary sampling rules. Thus 120 method associations are 40 independent traces. The frozen duplicate rule gives an identical control artifact one owner, the lowest ordered eligible group. No prospective duplicates or unscorable delivered carriers occurred.

4.2 GPU Execution and Timing

All inference runs serially on one NVIDIA RTX 5000 Ada Generation GPU with 32 GB memory and driver 590.48.01. Text uses the QuantFactory Llama 3 8B Instruct Q4_K_M GGUF and embedded tokenizer. Explicit `n_gpu_layers=-1` is verified as 33/33 eligible layers offloaded, with `logits_all=True` and batch/microbatch one. Context/KV state is reset between sequences and evaluation is incremental within a sequence. No CPU or partial-offload fallback is permitted.

PixelCNN++ uses a pretrained CIFAR-10 checkpoint with five residual blocks, 160 filters and ten mixtures. Strict loading, CUDA parameter/input placement and evaluation/inference modes are verified. The float32 network uses neither TF32 nor mixed precision. Deterministic algorithms, disabled cuDNN benchmarking, synchronous launches and fixed cuBLAS workspace are retained. Both backends have positive process-specific GPU activity evidence. Full hashes and settings are in the companion.

Prospective timings use the reference image path, not the later graph optimization. Charged latency spans child-process launch to exit, including imports, hashing/loading, occupied CPU work, inference, serialization and teardown. Encode/decode phase times exclude some overhead. CUDA events are nested within phases and are not added again. Fresh loading does not imply eviction of filesystem caches. The separate evaluator and wrapper monitoring joins are outside this boundary.

4.3 Metrics and Grouped Uncertainty

For L source bytes and N delivered symbols, exact goodput is $G = 8L/N$ only for an authenticated, exact and conforming recovery, and zero otherwise. Text uses bits/token and PNG bits/channel value; multiplying the latter by three gives bits/pixel. Complete-packet rate counts 2,336 bits through the packet stop or across the full carrier. It is undefined for incomplete A1 text, whose prefix progress is reported separately.

Recovery uses two-sided 95% Clopper–Pearson intervals, [0.8316, 1] for 20/20 and [0, 0.1684] for 0/20. These are Bernoulli working-model bounds for a small fixed allocation. Continuous quantities, AUCs and paired differences use 2,000 frozen PCG64 draws, base seed 2026090901 plus direction offset. Whole groups are resampled within image class or text-length band, carrying all methods and shared controls together. Intervals use the 2.5th and 97.5th percentiles. Boundary paired-binary comparisons additionally retain conservative bounds from 97.5% marginal Clopper–Pearson intervals, rather than interpreting collapsed bootstrap intervals as certainty.

4.4 Observer and Context Mismatch

The observer knows the exact model, profile, eligibility and scoring rules and, initially, the correct context, but not the key or source. Its frozen whole-carrier

scores are

$$S(y;c) = -N^{-1} \sum_t \log_2 q_t(y_t), \tag{7}$$

$$R(y;c) = N^{-1} \sum_t \log_2 \mathrm{rank}_t(y_t). \tag{8}$$

Higher always indicates stego. AUC is the probability that a stego score exceeds an ordinary score, with half weight for ties. No score orientation or threshold is selected using held-out labels. Scorable failed carriers remain included.

The extension retains correct-row scores and rescores the same 60 stego PNGs and 20 shared controls under row2. This random-RGB development row was frozen using PCG64 seed 2002, not detection outcomes. The scorer reads only PNG, profile and row and evaluates every delivered channel. Three predetermined development artifacts, fixed, A1 and ordinary PNG, first reproduced their retained scores and both available probability-stream digests. Paired AUC changes use 2,000 length-stratified draws, seed 2026090902, carrying both conditions, all methods and shared controls together. They are repeated measurements, not new transmissions. The subsequent within-artifact score-shift summaries are descriptive and post hoc, with no added tests or intervals.

4.5 Matched Development Benchmark

The lowest ordered development payload in each of three byte bands gives 32-, 96- and 128-byte messages under row1. Fixed rank has three matched timing repetitions per payload with alternating reference/graph order. The 32-byte case also tests gated and A1 once, giving 11 comparisons and 22 recoveries, not 11 independent payloads.

The graph path captures the unchanged PixelCNN++ forward after three warmups and replays from persistent input/output buffers. Both modes use fresh senders and receivers and the same retained packet. Equivalence requires exact eligible identifiers, ordering, float64 probabilities, PNG bytes, recovered bytes and completion outcomes. Headline speedup divides matched mean charged pair times over the nine fixed comparisons. Throughput pools recovered source bytes over both processes. No numerical tolerance is introduced.

5 RESULTS

5.1 Exact Artifact Recovery and Useful Rate

The prospective allocation completed all 120 stego outcomes and 40 independent controls. There were 100 exact, authenticated and conforming recoveries and 20 retained A1 text capacity failures. All 280 model processes have GPU execution evidence. No source replacement, protocol retry, timeout or infrastructure failure was needed. Table 2 and Figure 2 retain every attempt.[5]

Fixed and gated coding recover 20/20 payloads through text and 20/20 through PNG. Fixed text uses 584 packet positions and 32 completion tokens, giving $2,048/616 = 3.32468$ useful bits per token for the 256-byte image source. Gating adds a mean 39.8 skipped positions, lowering goodput to 3.12464 bits per token. The paired change relative to fixed is -0.2000 bits per token, with interval $[-0.2192, -0.1805]$. Gating also increases mean charged pair time by 23.96 seconds, with interval [20.34, 27.58]. It reduces whole-carrier mean surprisal by 0.3912 bits per token [0.3079, 0.4764].

All three PNG methods recover 20/20 messages. Their fixed full canvas gives the same mean exact goodput, 0.25054 bits per channel value, or 0.75161 bits per pixel. The 95% grouped interval is [0.24731, 0.25349] bits per channel. Equality of these rates is partly a design consequence, not evidence that the coders have identical capacity. Fixed coding uses 584 packet-bearing channels followed by 2,392 completion channels. Gating adds a mean 100.5 skips before the packet stops, replacing some ordinary completion within the same delivered size. A1 has a mean packet span of 597.85 channels, including 89.65 zero-bit positions, then completes the canvas.

Each packet adds 36 framing/encryption bytes. Image sources fill the slot; texts leave 162.8 padding bytes on average. Complete-carrier packet rates are 3.79221 bits/token for fixed text, 3.56404 for gated text and 0.78495 bits/channel for PNG. They include overhead and are not useful-source rates. Expanded framing, serialized expansion and termination accounting are in the companion.

5.2 Arithmetic Text Reaches the Capacity Boundary

All 20 A1 text streams reach the 2,048-token cap before producing the 2,336-bit packet. They recover be-

[5] Drafting used Codex (OpenAI, 2025).

Table 2: Prospective outcomes and original reference-path runtime. Each cell has 20 attempted payload groups. Exact recovery is 20/20 with 95% interval [83.16%, 100%], or 0/20 with [0%, 16.84%]. Phase means exclude loading and some process overhead. Charged means include both fresh processes and unsuccessful attempts. Goodput units are bits/token for text and bits/channel value for PNG.

Carrier	Method	Exact	Goodput	Encode (s)	Decode (s)	Charged (s)	Failure
UTF-8	Fixed	20/20	3.3247	45.63	45.31	123.33	None
UTF-8	Gated	20/20	3.1246	57.31	57.30	147.29	None
UTF-8	A1	0/20	0.0000	398.15	396.01	826.57	Capacity
PNG	Fixed	20/20	0.2505	85.27	85.31	178.18	None
PNG	Gated	20/20	0.2505	85.29	85.65	178.53	None
PNG	A1	20/20	0.2505	85.59	85.77	179.01	None

tween 581 and 2,216 packet-prefix bits, with mean 1,456.85. None reaches zero-extension or finite-message termination. Accordingly, none authenticates or delivers useful source bytes, despite the availability of partial information. Mean charged encode-plus-decode time is 826.57 seconds, over six times fixed text, and all that cost remains in the results.

Independent interval and partition checks agree with all 40 prospective arithmetic audits. Sender/receiver prefix and source-window checks also agree. All 20 arithmetic PNG packets terminate and recover exactly. Two failed text streams have isolated unchanged intervals, one and three steps respectively, with longest consecutive run one. A successful PNG has one such step as well. These observations do not support sustained stagnation as the cause of the text deficits. Zero-bit runs must not be confused with unchanged intervals, since an interval may continue narrowing without yet sharing a leading bit.

Cumulative surprisal closely tracks recovered text-prefix information. The best-supported explanation is low information along these trajectories under the fixed cap, not decoder disagreement or an unexecuted flush. This observed capacity limit is distinct from A1's separately demonstrated stagnation case.

5.3 Detectability with the Correct Context

Correct-context scores show that exact recovery and low detectability are different endpoints. Both fixed and gated text have empirical mean-surprisal AUC 1.000, with bootstrap interval [1.000, 1.000]. Every observed stego score exceeds the corresponding ordinary sample range. This is complete empirical separation under the tested observer, not an assurance of perfect future detection. Failed A1 text gives AUC 0.380 [0.260, 0.503]. The below-chance orientation is retained rather than reversed to improve a detector result.

PNG scores are less separated. Fixed mean-surprisal AUC is 0.8850 [0.7425, 0.9850], gating 0.8300 [0.7000, 0.9375], and A1 0.5575 [0.3750, 0.7426]. Arithmetic PNG combines exact delivery with lower observed separation for this score, although its interval spans a wide range around chance. Every cell has 20 scorable stego artifacts and 20 unique matched controls, without exclusions. The companion reports individual observations and both frozen scores.

5.4 PNG Detection Depends on the Tested Context

All 80 retained PNGs are fully scorable under row2. Fixed-rank primary AUC decreases from 0.8850 to 0.6975, a paired change of -0.1875 with interval $[-0.2775, -0.1000]$ (Figure 3). Gating changes from 0.8300 to 0.7475, or -0.0825 [-0.1625, 0.0100]. A1 changes from 0.5575 to 0.5925, or $+0.0350$ [-0.0225, 0.1050]. The fixed reduction is resolved by its paired interval. The gated and A1 changes remain uncertain.

Under mismatch, the primary fixed interval [0.5150, 0.8700] only narrowly excludes 0.5, and its secondary rank-score interval [0.4725, 0.8475] includes 0.5. The primary gated interval [0.6074, 0.8825] remains above chance. Thus this mismatch weakens the tested fixed score without removing all information from mean-surprisal detection. A resolved change for one method and an uncertain change for another do not themselves establish different method sensitivities.

Descriptive within-artifact mean score increases are 0.610 for controls, 0.334 for fixed, 0.513 for gated and 0.629 for A1, in bits per delivered channel. The paired mean fixed-minus-control gap narrows from 0.508 to 0.231. Larger control movement is consistent with the narrower gap. These descriptive post hoc means do not establish why AUC changed, since AUC depends on all score rankings. Row1 comes from a generated image and row2 from random RGB values, so the comparison does not isolate context correctness

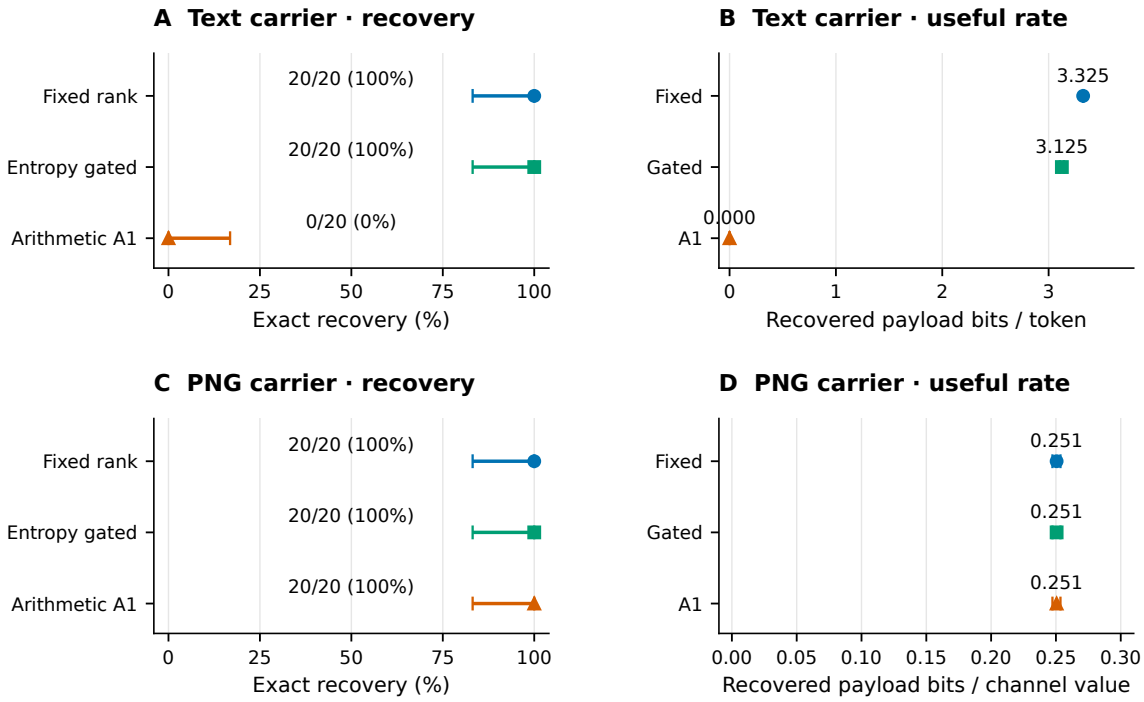

Figure 2: Held-out recovery and exact useful rate for 20 payload groups per direction. Recovery whiskers are the accepted 95% Clopper–Pearson bounds. Rate whiskers are the accepted stratified group-bootstrap intervals, keeping paired methods together. All failed A1 text transmissions contribute zero goodput. Text and PNG rate axes use different carrier symbols. Zero-width observed rate intervals do not imply universal reliability.

from row statistics.

5.5 Exact GPU Replay Improves Useful Throughput

CUDA graph replay reduces mean fixed-rank charged encoding from 88.95 to 17.36 seconds and decoding from 88.87 to 17.28 seconds. Mean combined latency falls from 177.8186 to 34.6417 seconds, a matched 5.133-fold improvement. Pooled useful throughput rises from 0.4799 to 2.4633 recovered bytes per second. These estimates include all nine fixed timing comparisons and both fresh processes, not just network execution. Gated and A1 have one matched 32-byte comparison each, with combined times changing from 180.25 to 34.61 and 182.85 to 35.11 seconds respectively. Those two measurements do not establish broader method-specific performance distributions.

All 11 reference/graph comparisons match the exact eligible identifiers, rank ordering and float64 probability streams across 2,976 channel positions, including completion. All 22 saved PNGs recover identical source bytes. No tolerance, altered precision or rerolled packet is involved. This supports retaining graph replay as an explicit optional execution mode on the tested stack, with reference execution still available. It does not retroactively alter prospective timings.

Peak allocated tensor memory decreases from 484.48 to 473.85 MiB, but reserved memory rises from 516 to 744 MiB and sampled process memory from 912 to 1,176 MiB. The companion separates loading and nested CUDA-event/phase intervals without double counting. Thus latency improves at greater reservation/process-memory cost.

Prospective execution costs 42,024.53 charged seconds, including failures and controls. The benchmark costs 2,351.73 seconds and context rescoring 1,439.65 seconds. Including earlier development, total measured use is 24.72 GPU-hours. Occupied-process costs are not energy measurements.

6 DISCUSSION AND LIMITATIONS

The shared contract reveals how serialization, finite capacity and completion govern authenticated recov-

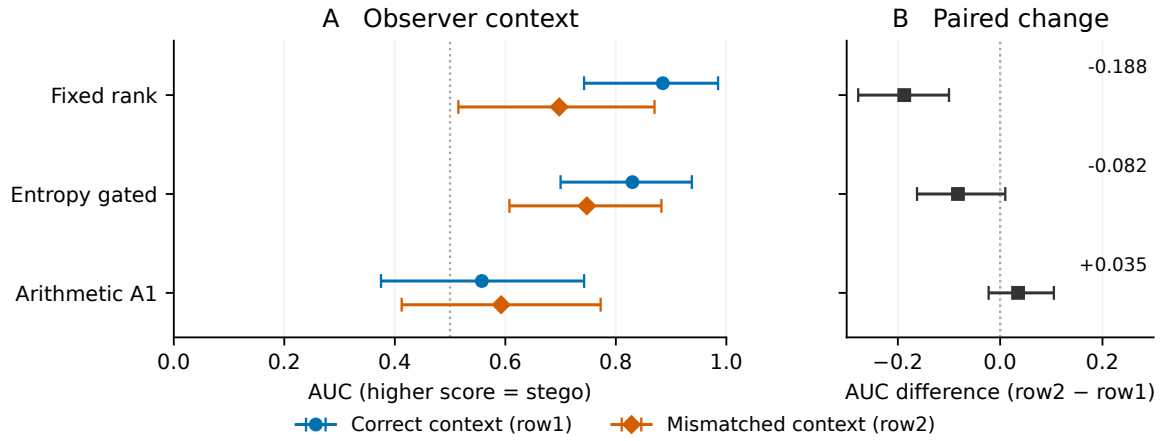

Figure 3: Primary PNG detection with correct row1 and mismatched row2. The same 60 stego PNGs and 20 shared ordinary controls appear in both conditions. Panel B gives paired AUC change, row2 minus row1. Whiskers use the accepted 2,000 length-stratified group draws, jointly retaining observer conditions, methods and control ownership. The observer still knows the exact model and procedure. Row2 is the previously frozen random-RGB row, not a row selected for low detection. Reference lines are not fitted classification thresholds.

ery. Rank coding supplies text capacity but a strong known-model detection signal. A1 succeeds for PNG yet fails on low-information text trajectories. These outcomes connect coder choices to the observable artifact rather than nominal alphabet size alone.[6]

Development isolates serialization. Twenty image payloads under two prompts yield 40 packet-paired sequence/static comparisons. Complete-prefix filtering recovers 40/40, versus 22/40 for singleton filtering, split 8/20 for the forest prompt and 14/20 for soup. All 18 drift failures remain unrepaired artifacts. This supports the saved-text consistency requirement, alongside published stepwise verification, across 20 distinct payloads rather than 40 independent sources. Twenty lossless development PNG re-saves also preserve pixels and recover exactly. These checks do not cover JPEG, cropping or paraphrasing.

The study uses 20 held-out groups per direction and two pinned models. Shared controls and repeated context scores retain their group dependence, and exploratory comparisons are not family-wise confirmatory tests. Neither near-chance AUC nor withholding a conditioning row establishes cryptographic secrecy or resistance to arbitrary observers. Detecting a carrier is also distinct from recovering its encrypted contents. A1's capacity outcomes and documented stagnation limitation concern this adaptation, not all arithmetic or range coders.

Graph replay addresses image-model launch overhead. Text filtering still consumes approximately 71%, 75% and 89% of fixed, gated and A1 encoding time. Faster GPU inference cannot remove that CPU bottleneck or supply missing information under the frozen cap. The measured 5.13-fold improvement covers three development payloads on the pinned stack. Exact replay and speed across other libraries or GPU architectures remain unverified.

Reproduction also depends on dissemination terms. The image port has a sale-restricting notice, and separate checkpoint redistribution permission was not established. No weights, keys or private row bytes are distributed with the manuscript.

7 CONCLUSION

We develop and evaluate exact bidirectional steganographic transport using a language model for text carriers and a pixel autoregressive model for PNG carriers. A common authenticated packet, observable-symbol interfaces and independent saved-file recovery enable a controlled comparison across modalities. Fixed and gated coding recover all held-out payloads in both directions. Arithmetic A1 recovers all PNG payloads but exposes a clear text capacity boundary under the frozen cap.[7]

The resulting evidence also separates recovery from concealment and execution cost. Known-model

[6] Drafting used Codex (OpenAI, 2025).

[7] Drafting used Codex (OpenAI, 2025).

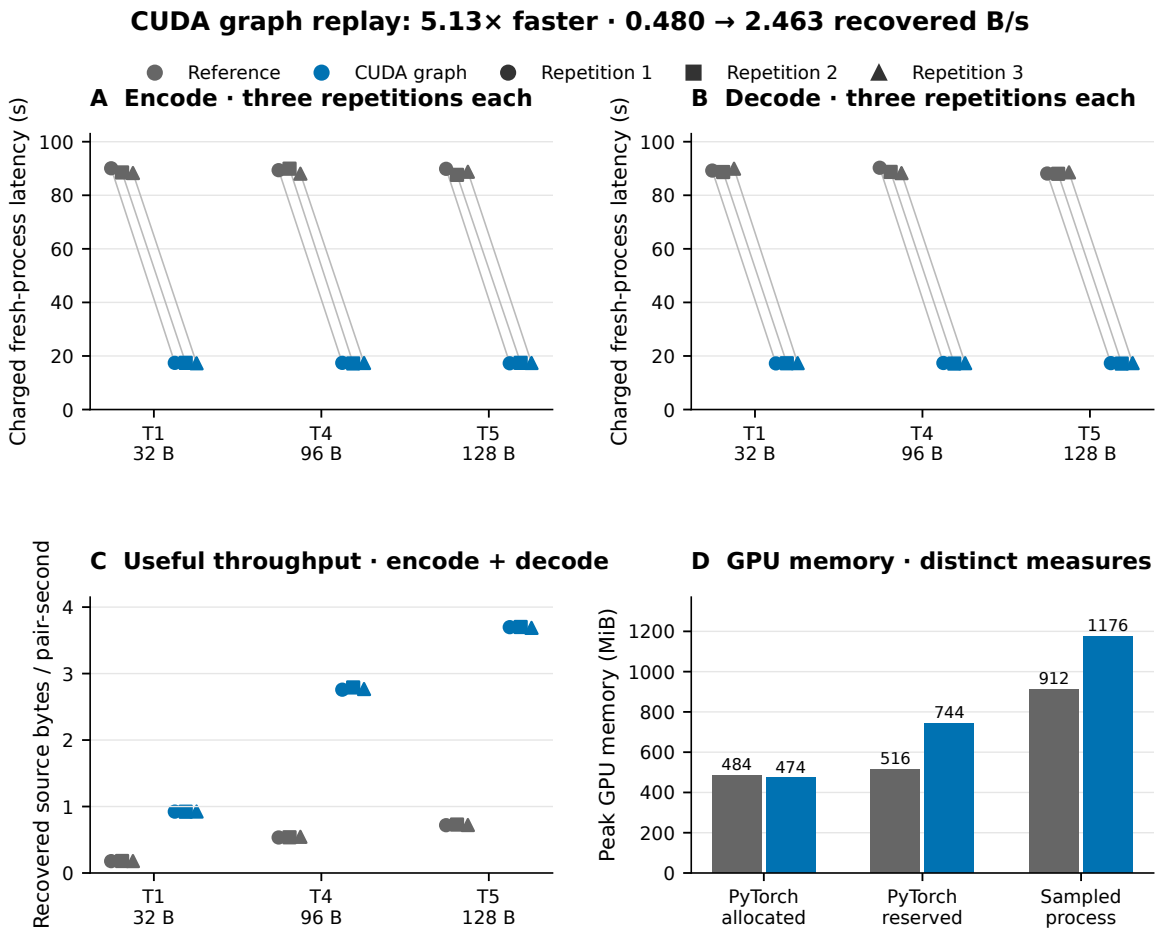

Figure 4: Development GPU benchmark on the pinned RTX 5000 Ada configuration. Fixed rank has three alternating-order matched repetitions for each of three payloads, not nine independent scientific payloads. Encode/decode latency includes fresh-process loading, graph setup and teardown. Throughput counts recovered source bytes over both processes. All 11 comparisons, including the additional single-payload gated/A1 checks, preserve exact probability streams, PNG bytes and recovery. Allocated, reserved and sampled process memory are overlapping measures and must not be summed.

scores detect rank-coded text strongly, while one specified image-context mismatch reduces fixed-rank AUC with uncertain changes for the other methods. A matched CUDA graph benchmark improves useful image throughput by 5.13 times while retaining exact checked probabilities and artifacts. The central finding is that reliable artifact transport, distributional behavior and practical throughput must be evaluated together. None can be inferred from invertible internal ranks or plausible generated appearance alone.

AI ASSISTANCE DISCLOSURE

OpenAI Codex assisted with implementation, analysis scripts, figure preparation, and drafting and editing throughout this paper and its companion (OpenAI, 2025). Reported measurements come from retained execution records. Statistical plots display those measurements, and carrier examples are saved experimental artifacts, not AI-generated replacements. Tool assistance does not constitute independent scientific verification. Responsibility for factual accuracy, attribution and the final submission rests with the authors.

REFERENCES

Dworkin, M. (2007). Recommendation for block cipher modes of operation: Galois/Counter mode (GCM) and GMAC. Technical Report SP 800-38D, National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-38D.

Gehrmann, S., Strobelt, H., and Rush, A. M. (2019). GLTR: Statistical detection and visualization of generated text. In *Proceedings of ACL: System Demonstrations*, pages 111–116. Association for Computational Linguistics. https://doi.org/10.18653/v1/P19-3019.

Mantzaris, A. V. (2026a). LlmStenoExplore. Software repository and empirical exploration. Revision b9f1650. https://github.com/mantzaris/LlmStenoExplore.

Mantzaris, A. V. (2026b). RankCloak conceals the surface form of synthetic cryptographic artifacts in language model generated text. Repository manuscript and implementation. Revision ce853d4. https://github.com/mantzaris/llm-rankcloak.

Meta (2024). Meta Llama 3 model card. Official model documentation. https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.

Norelli, A. and Bronstein, M. (2026). LLMs can hide text in other text of the same length. arXiv:2510.20075v6. Original preprint 2025. https://arxiv.org/abs/2510.20075v6.

OpenAI (2025). Introducing Codex. Official product description, 16 May. https://openai.com/index/introducing-codex/. Tool attribution, not a scientific evidence source.

PyTorch Contributors (2024). CUDA semantics. PyTorch 2.5 documentation, CUDA graphs. https://github.com/pytorch/pytorch/blob/v2.5.1/docs/source/notes/cuda.rst.

Salimans, T., Karpathy, A., Chen, X., and Kingma, D. P. (2017). PixelCNN++: Improving the PixelCNN with discretized logistic mixture likelihood and other modifications. arXiv:1701.05517. https://arxiv.org/abs/1701.05517.

Schroeder de Witt, C., Sokota, S., Kolter, J. Z., Foerster, J., and Strohmeier, M. (2023). Perfectly secure steganography using minimum entropy coupling. In *International Conference on Learning Representations*. https://arxiv.org/abs/2210.14889.

Xiao, H., Rasul, K., and Vollgraf, R. (2017). Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. arXiv:1708.07747. https://arxiv.org/abs/1708.07747.

Yan, R. and Murawaki, Y. (2025). Addressing tokenization inconsistency in steganography and watermarking based on large language models. In *Proceedings of EMNLP*, pages 7076–7098. Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.emnlp-main.361.

Yan, R. and Murawaki, Y. (2026). Efficient provably secure linguistic steganography via range coding. In *Proceedings of ACL (Volume 1: Long Papers)*, pages 890–907. Association for Computational Linguistics. https://doi.org/10.18653/v1/2026.acl-long.39.

Zhang, S., Yang, Z., Tu, H., Yang, J., and Huang, Y. (2021). Pixel-Stega: Generative image steganography based on autoregressive models. arXiv:2112.10945. https://arxiv.org/abs/2112.10945.

Ziegler, Z. M., Deng, Y., and Rush, A. M. (2019). Neural linguistic steganography. In *Proceedings of EMNLP-IJCNLP*, pages 1210–1215. Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1115.